

Шпаргалка к защите Лабораторной работы №1

Основи нереляційних баз даних NoSQL | Преподаватель: ас. Жишкович А. В.

Лекции 01, 02, 03 (Вступ, Розподілені БД, Типи NoSQL)

1. Почему классический SQL перестал справляться (Предпосылки NoSQL)

Всю историю IT реляционные базы (PostgreSQL, Oracle, MySQL) были идеальны: они хранят данные в таблицах и гарантируют 100% точность транзакций по стандарту ACID. Но в конце 2000-х годов с развитием смартфонов и соцсетей начался бум Великих Данных (Big Data).

Концепция Big Data: классические 3V и расширенные 7V (Слайд 11)

Классически Big Data определили через 3V (Дуг Лейни, Gartner). Именно эти 3 технических фактора сломали SQL и породили NoSQL (требование Задания №2 в лабе):

- **1. Volume (Обсяг):** Терабайты и петабайты. Данные не влезают на один сервер (пример: Netflix — терабайты телеметрии ежедневно в Cassandra).
- **2. Velocity (Швидкість):** Миллионы событий в секунду. Блокировки SQL зависают (пример: Instagram — миллионы лайков в секунду в RocksDB/Redis).
- **3. Variety (Різноманітність):** Гетерогенные форматы без единой схемы (пример: Amazon — миллионы товаров разных категорий в DynamoDB).

В Слайде 11 лекции преподаватель расширяет модель до концепции «7V» (бизнес-анализ данных):

- **4. Value (Цінність):** Мало собрать данные, нужно извлечь из них пользу и монетизировать для бизнеса.
- **5. Veracity (Достовірність):** Качество и правдивость данных. Ошибочные/зашумленные данные ведут к неверным решениям.
- **6. Variability (Мінливість):** Контекст и значение данных могут быстро меняться со временем.
- **7. Visualization (Візуалізація):** Понятное визуальное представление (графики, дашборды, тепловые карты) для руководства.

Два тупика традиционного SQL

1. Масштабирование: SQL создан для работы на одном сервере. Увеличивать его мощность (Scale-Up / купить дороже процессор и RAM) безумно дорого, и существует физический потолок материнской платы. А горизонтально разделить таблицы со связями Foreign Key на 20 компьютеров почти невозможно: тяжелые операции JOIN по сети работают чудовищно медленно.

2. Жорсткая схема (Schema-on-Write): В маркетплейсе у миллиона товаров разные свойства. В SQL изменение структуры (ALTER TABLE) на терабайтной таблице блокирует сайт на несколько часов. Это миллионные убытки.

💡 История термина NoSQL (вопрос с подвохом!)

- 1998 год: Карло Строчи (Carlo Strozzi) впервые ввел термин NoSQL для своей реляционной базы без SQL-интерфейса.
- 2009 год: Йохан Оскарссон (Johan Oskarsson) возродил термин на митапе разработчиков распределенных баз.
- Значение сегодня: Not Only SQL ('Не лише SQL') — признание того, что для разных задач нужны

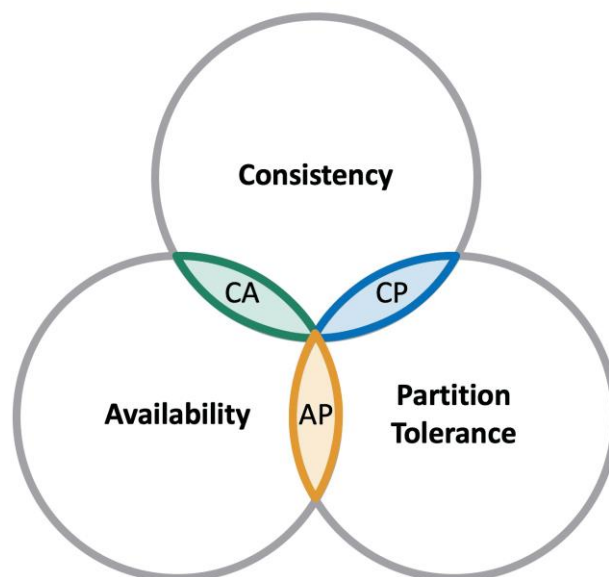
2. Теорема CAP (Брюер, 2000; Гилберт и Линч, 2002)

Главный закон распределенных систем. Утверждает, что распределенная система может гарантировать только 2 из 3 свойств:

- **C (Consistency / Узгодженість):** Все серверы в один и тот же момент времени видят строго одинаковые, самые свежие данные.
- **A (Availability / Доступність):** Каждый запрос пользователя всегда получает успешный ответ без ошибок.
- **P (Partition Tolerance / Стійкість до розщеплення):** Система продолжает жить, даже если кабель сети между серверами физически перебит.

💡 Почему формула «Выбери 2 из 3» — это миф?

В реальном физическом мире сетевые сбои (Partition / P) неизбежны. Кабель перерубят, свитч зависнет. Вы не можете выключить вероятность сбоя сети. Поэтому система CA в кластере невозможна. Реальный выбор инженера всегда между двумя вариантами: CP или AP!



Два сценария препода: Банк (CP) против YouTube (AP)

- Банковские переводы -> Модель CP (Точность понад усе):

Если сеть между серверами Киева и Львова порвалась, банкомат ОБЯЗАН ОТКАЗАТЬ в выдаче денег ('Сервис временно недоступен'). Если бы банк выбрал доступность (AP), мошенник снял бы последние 1000 грн в Киеве и тут же снял бы их во Львове (двойная трата). Банку выгоднее временный отказ, чем финансовые убытки.

- Комментарии на YouTube -> Модель AP (Доступность понад усе):

Если связь между дата-центрами упала, YouTube ОБЯЗАН ПРИНЯТЬ комментарий и показать

пользователю 'Опубликовано'. Пусть даже в другой стране этот комментарий увидят на 3 секунды позже — это никому не навредит. Главное — сайт никогда не падает.

3. Сравнение моделей ACID и BASE. Eventual Consistency

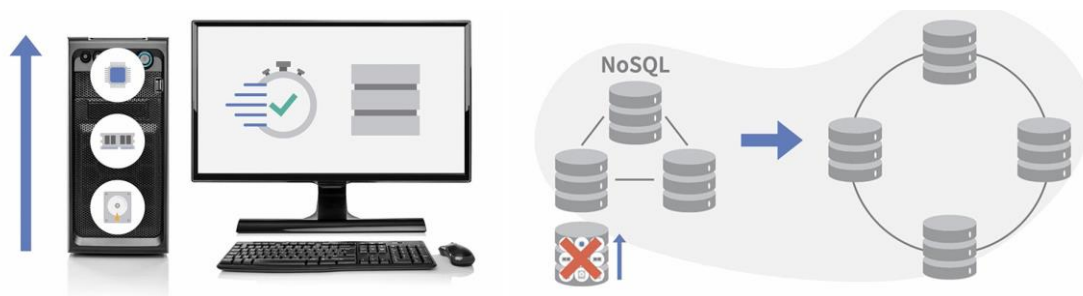
Это две противоположные философии разработки:

Параметр	ACID (Классический SQL)	BASE (Распределенный NoSQL)
Философия	Пессимистичная: строгость и точность превыше всего. Лучше заблокировать базу, чем сохранить ошибку.	Оптимистичная: доступность и скорость превыше всего. Сервис обязан отвечать всегда.
1-я буква	Atomicity (Атомарность): транзакция прошла вся целиком или полностью откатилась (Rollback).	Basically Available (Базовая доступность): ответ будет дан всегда, даже при падении узлов.
2-я буква	Consistency (Узгодженість): данные строго соответствуют всем правилам и внешним ключам.	Soft state (Мягкое состояние): данные меняются сами в фоне за счет асинхронной репликации.
3-я буква	Isolation (Изолированность): параллельные операции не видят промежуточных состояний друг друга.	Eventual consistency (Узгодженість у кінцевому підсумку): через пару секунд все реплики станут равны.
4-я буква	Durability (Долговечность): если операция подтверждена, она железно сохранена на диске.	- (полагается на копирование данных на несколько серверов)

💡 Eventual Consistency: почему Facebook можно, а банку нельзя?

Eventual Consistency гарантирует: если новые данные не поступают, через короткое время реплики синхронизируются. Для Facebook задержка в 2 секунды незаметна: автор запостил фото, а друг в США увидел его через 2 секунды. Никто не пострадал, зато сервер не блокировался. В банке задержка в 2 секунды позволяет потратить одни и те же деньги дважды!

4. Масштабирование: Scale-Up против Scale-Out. Шардинг и Репликация



- **Scale-Up (Вертикальное):** Покупка одного все более дорогого суперсервера. Быстро упирается в потолок разъемов материнской платы. Дорого, единая точка отказа.
- **Scale-Out (Горизонтальное):** Объединение в кластер десятков дешевых серверов. Выросла нагрузка — просто докупили 3 обычных системника. Идеально для NoSQL.

- **Шардинг (Sharding):** Разрезание базы на куски (шарды) по Shard Key (например, пользователи А-М на сервер 1, N-Z на сервер 2). Позволяет горизонтально масштабировать запись.
- **Репликация (Replication):** Копирование ОДНИХ И ТЕХ ЖЕ данных на несколько серверов для надежности (Master-Slave: пишет Master, читают Slaves; Multi-Master: пишут все).

5. 5 основных типов NoSQL баз данных (Лекция 03)

1. Ключ — Значение (Key-Value): Redis, Memcached, DynamoDB

Самый быстрый тип (сложность $O(1)$). Как гигантский словарь в памяти: ключ -> байты. Redis держит данные в RAM и отвечает быстрее 1 миллисекунды. Применение: кэш, сессии пользователей, счетчики в реальном времени, корзина интернет-магазина.

2. Документно-ориентированные (Document): MongoDB, Couchbase

Хранят данные в виде полуструктурированных документов (JSON/BSON). Внутри документа могут быть списки и вложенные объекты. Применение: каталоги маркетплейсов с разнородными товарами, карточки пользователей, блоги, мобильные приложения.

3. Ширококолонковые (Wide-Column / Column-Family): Apache Cassandra, HBase

Данные группируются по семействам колонок. Строки могут содержать миллионы колонок разного состава. Невероятная скорость параллельной записи. Применение: телеметрия датчиков IoT, логи, финансовые временные ряды, история просмотров в Netflix.

4. Графовые (Graph): Neo4j, Amazon Neptune

Хранят вершины (Nodes), связи (Edges) и свойства. Переход по связям происходит по указателям в памяти $O(1)$ без тяжелых JOIN. Применение: социальные сети (друзья друзей), рекомендательные системы, антифрод (выявление цепочек мошенников), карты и логистика.

5. Векторные (Vector DB — главный тренд 2024-2026): Pinecone, Milvus, Qdrant

Специальные базы данных эпохи AI и LLM. Хранят не текст, а многомерные векторы (эмбединги из 1536 чисел), сгенерированные нейросетями. Ищут похожие смыслы по косинусному расстоянию. Применение: RAG-системы чат-ботов, поиск похожих картинок, семантический поиск.

6. Polyglot Persistence (Багатоваріантна персистентність)

Концепция Мартина Фаулера (2011): в современной корпоративной системе не существует одной идеальной базы данных. Каждый микросервис берет ту базу, которая лучше всего решает его узкую задачу.



Пример реальной архитектуры (Сервис такси уровня Uber / Bolt)

- Redis (Key-Value) -> держит в RAM текущие GPS-координаты 100 000 водителей (обновление каждую секунду, отклик < 1 мс);
- MongoDB (Document) -> хранит карточки водителей, машин и тарифы (разнородные документы);
- Neo4j (Graph) -> строит оптимальный маршрут по графу улиц с учетом развязок;
- PostgreSQL (SQL/ACID) -> списывает деньги с карты и начисляет баланс (строгий финансовый учет);

- Apache Cassandra (Wide-Column) -> пишет гигабайты логов поездок для аналитики.

7. Анализ мирового рейтинга DB-Engines (Сентябрь 2026)

На реальном скриншоте рейтинга db-engines.com/en/ranking четко видны главные законы индустрии:

- **Первые 4 места:** Oracle (1122), MySQL (845), MS SQL Server (699), PostgreSQL (683). Это реляционные базы. Вывод: появление NoSQL не убило SQL. Реляционные базы незаменимы для финансов и учета.
- **№5 в мире и №1 среди NoSQL:** MongoDB (381 балл). Лидер с огромным отрывом за счет удобства работы с документами JSON.
- **№8 в мире и №1 в Key-Value:** Redis (158 баллов). Главный инструмент кэширования в интернете.
- **№11 и №12:** Elasticsearch (95.5) и Apache Cassandra (94.0). Лидеры полнотекстового поиска и записи телеметрии.
- **№18 и №20:** Amazon DynamoDB (56) и Neo4j (47, №1 среди графовых).

Rank	Rank			DBMS	Database Model	Score		
	Sep 2026	Aug 2026	Sep 2025			Sep 2026	Aug 2026	Sep 2025
1.	1.	1.	1.	Oracle	Relational, Multi-model	1122.27	-1.17	-48.36
2.	2.	2.	2.	MySQL	Relational, Multi-model	844.82	+2.50	-46.95
				Microsoft SQL Server	Relational, Multi-model	698.69	+4.13	-18.63
3.	3.	3.	3.	PostgreSQL	Relational, Multi-model	683.26	-1.32	+26.08
4.	4.	4.	4.	MongoDB	Document, Multi-model	381.31	-3.57	+0.81
5.	5.	5.	5.	Snowflake	Relational	211.61	-4.28	+21.42
6.	6.	6.	6.	Databricks	Multi-model	168.34	+2.53	+44.27
7.	7.	7.	7.	Redis	Key-value, Multi-model	158.06	+1.55	+12.89
8.	8.	8.	8.	IBM Db2	Relational, Multi-model	110.57	-1.50	-13.61
9.	9.	9.	9.	SQLite	Relational	97.05	+1.11	-10.83
10.	10.	10.	10.	Elasticsearch	Multi-model	95.50	+1.16	-22.75
				Apache Cassandra	Wide column, Multi-model	93.96	-6.02	-13.02
11.	11.	11.	11.	MariaDB	Relational, Multi-model	74.42	-4.65	-17.04
12.	12.	12.	12.	Splunk	Search engine	71.68	+0.45	-4.09
				Microsoft Azure SQL Database	Relational, Multi-model	70.84	+0.08	-8.34
13.	13.	13.	13.	Apache Hive	Relational	70.11	-0.97	-5.98
14.	14.	14.	14.	Microsoft Access	Relational	63.06	-1.24	-20.55
				Amazon DynamoDB	Multi-model	56.36	+2.16	-23.92
15.	15.	15.	15.	Google BigQuery	Relational	52.58	-0.12	-13.42
16.	16.	16.	16.	Neo4j	Graph	47.17	-2.42	-6.61

8. Блиц-ответы на 5 главных коварных вопросов на защите

Вопрос 1: Какую базу выбрать для авиабилетов, а какую для интернет-магазина?

Ответ: Для авиабилетов — только реляционный SQL (PostgreSQL/Oracle). Мест в самолете ограниченное количество, овербукинг и гонки транзакций недопустимы, нужен строгий уровень изоляции ACID.

Для интернет-магазина — документоориентированный NoSQL (MongoDB). У товаров миллионы разных характеристик (у телефона процессор, у обуви размер). В MongoDB каждый товар хранится как отдельный JSON без переделывания схемы всей базы.

 Вопрос 2: Объясните теорему CAP на примере банка и YouTube.

Ответ: При неминуемом сетевом обрыве между серверами (P) система выбирает: либо точность (C), либо доступность (A).

Банк выбирает CP: если кабель между Киевом и Львовом перебит, банкомат отказывает в операции. Лучше показать ошибку, чем позволить снять одни деньги дважды.

YouTube выбирает AP: сервер принимает комментарий и пишет 'Опубликовано'. Пусть другие пользователи увидят его на 3 секунды позже после починки сети — главное, сервис работает без сбоев.

 Вопрос 3: В чем разница между Scale-Up и Scale-Out?

Ответ: Scale-Up (вертикальное) — покупка одного суперкомпьютера с 128 ядрами и 2 ТБ RAM. Дорого и есть физический потолок материнки.

Scale-Out (горизонтальное) — добавление в кластер десятков дешевых стандартных серверов. Данные делятся шардингом и кластер растет без ограничений.

 Вопрос 4: Что такое Eventual Consistency?

Ответ: Гарантия модели BASE, что при отсутствии новых изменений все копии данных на репликах через короткое время (секунды) станут одинаковыми. Для ленты соцсетей это идеально, для баланса банковской карты — недопустимо.

 Вопрос 5: Что такое Polyglot Persistence?

Ответ: Концепция Мартина Фаулера, когда в одном проекте разные задачи решают разные СУБД: Redis для кэша и GPS, MongoDB для товаров, Neo4j для маршрутов, PostgreSQL для платежей.